

Excitation, stabiliser and governor models — a proposal

Status: proposal. Nothing here is implemented yet.

This is a plan for giving G2ELin a library of the standard AVR/exciter, PSS and turbine-governor models, instead of the single hard-wired set it has today. It says what exists now, what I propose to add and why, the equations and block diagram of each proposed model, and the four architectural decisions that have to be made before any of it is written.

It also answers the question that prompted it: what GENROU and GENCLS are, and how they relate to the machine model already here.

1. Recommendation in one table

Twelve models, in three phases. The column that matters is the last one: every model in phase 1 and 2 exists in ANDES 2.0, which is already installed in this repo's environment and is already the cross-validation reference ([tools/compare_andes.py](#)). Every one of them can therefore be checked against an independent implementation rather than against my reading of a standard.

Kind	Model	States	Why this one	ANDES
Exciter	SEXS	2	The simplest realistic AVR. The sane default for a bus you have no data for.	✓ sexs.py
	EXST1 / ST1A	4	Static (thyristor) excitation — the dominant modern technology, and the model in your Kundur figure .	✓ exst1.py
	EXDC2 / IEEE1	4	DC commutator exciter with saturation. Legacy technology, but it is what most existing datasets contain. Closest to what G2ELin has now.	✓ exdc2.py , ieee1.py
PSS	PSS1A	5	The classical single-input stabiliser. A strict generalisation of G2ELin's present PSS.	— (see IEEEEST)
	IEEEEST	7	PSS1A plus a selectable input signal and a second-order lead-lag. Supersedes PSS1A if you only want one.	✓ ieeest.py
Governor	TGOV1	2	The most-used simple steam governor. A strict generalisation of G2ELin's present governor.	✓ tgov1.py
	IEEEG1	6	The most-used <i>detailed</i> steam governor: HP/IP/LP stages, optional cross-compound second shaft.	✓ ieeeg1.py
	HYGOV	4	Hydro, with penstock water inertia. Gives the non-minimum-phase response no linear governor can fake.	✓ hygov.py

Kind	Model	States	Why this one	ANDES
	GAST	3	Gas turbine, with the exhaust-temperature limit.	✓ <code>gast.py</code>

Phase 3, deliberately deferred — PSS2B (dual-input, integral-of-accelerating-power), ESST4B / AC8B (digital PID exciters), GGOV1 (modern general-purpose governor), PSS4B (multi-band). Rationale in §6. Note in particular that **ANDES 2.0 has no PSS2B**, so it is the one popular model on this list with no ready cross-validation partner — which is exactly why it should not be first.

2. What G2ELin has today

Worth writing down precisely, because three of the proposed models turn out to be *supersets* of what is already here, which makes them cheap.

`components/sm.py` builds one fixed 19-state machine:

Block	States	Count
Step-up transformer current	<code>i_gd, i_gq</code>	2
Stator flux	<code>psi_d, psi_q</code>	2
Rotor windings	<code>psi_fd, psi_1d, psi_1q, psi_2q</code>	4
Swing	<code>dw_r, theta</code>	2
Governor	<code>P_m</code>	1
PSS	<code>dw_1, v_1, v_2, v_pss</code>	4
AVR / exciter	<code>e_1, e_2, e_fd, e_3</code>	4

2.1 The governor as implemented

$$dP_m/dt = (P_{ref} - \Delta P - P_m) / T_G, \quad \Delta P = (\omega_r - \omega_{ref}) / m_p$$

One state: a droop $1/m_p$ into a first-order lag T_G . No limits, no turbine stages.

2.2 The PSS as implemented

$$\begin{aligned} d\Delta\omega_1/dt &= (\Delta\omega_r - \Delta\omega_1) / T_{LP} && \text{input low-pass} \\ dv_1/dt &= K_{PSS} \cdot (d\Delta\omega_1/dt) - v_1 / T_{HP} && \text{washout, gain folded in} \\ dv_2/dt &= (T_{1n} \cdot (dv_1/dt) + v_1 - v_2) / T_{1d} && \text{lead-lag 1} \\ dv_{pss}/dt &= (T_{2n} \cdot (dv_2/dt) + v_2 - v_{pss}) / T_{2d} && \text{lead-lag 2} \end{aligned}$$

Four states: an input filter, a washout and two lead-lags, driven by rotor speed deviation. This is the PSS1A structure with the second-order input filter degenerated to a single pole.

Note the idiom — a lead-lag $(1+sT_n)/(1+sT_d)$ is written using the *derivative of the upstream state* rather than by splitting out a feedthrough term. It works because dv_1/dt is available symbolically, it keeps every block strictly proper, and **every new block below should follow it**.

2.3 The exciter as implemented

```

de_1/dt = (E_t - e_1) / T_r      terminal voltage
transducer
de_2/dt = (K_a·(V_ref - e_1 - e_3 + v_pss) - e_2) / T_a  regulator
de_fd/dt = (K_e·e_2 - e_fd) / T_e      exciter
de_3/dt = (K_fd·(de_fd/dt) - e_3) / T_fd  rate feedback

```

Four states, with defaults $T_r=0.02$, $K_a=300$, $T_a=0.001$, $K_e=1$, $T_e=1e-4$, $K_{fd}=0.001$, $T_{fd}=0.1$.

Two things to flag:

K_e here is not IEEE's K_E . In IEEE DC-exciter models the exciter equation is $T_E \cdot dE_{fd}/dt = V_R - (K_E + S_E(E_{fd})) \cdot E_{fd}$ — K_E is the *self-excitation* constant, multiplying the output. Here it multiplies the *input*. With $K_e = 1$ the two coincide, which is why the default is 1, but a user who sets $K_e = 0.3$ from a datasheet will get the wrong model silently. Any renaming of this parameter is a breaking change to saved networks, so it should happen in the same release as the model library.

With the shipped defaults this is already a static exciter. $T_e = 1e-4$ and $T_a = 1e-3$ make both lags effectively instantaneous and $K_{fd} = 0.001$ makes the rate feedback vestigial, so the model behaves as a high-gain $K_a = 300$ proportional regulator — i.e. as ST1A, not as a DC exciter. The *structure* is DC-exciter-shaped; the *defaults* are static. Adding **EXST1** therefore mostly makes the existing behaviour honest.

2.4 What is missing, across all three

- **No limits anywhere.** No V_{RMAX}/V_{RMIN} , no V_{MAX}/V_{MIN} on the valve, no PSS output clamp. Every real model in §4 has them.
- **No saturation.** No exciter S_E , no machine saturation.
- **No choice of input signal** for the PSS (speed only).
- **No turbine.** The governor output is the mechanical power; there is no steam chest, reheater or penstock.

3. Your question: GENROU and GENCLS

These are **machine** models, not controllers — PSS/E's names, adopted by PowerWorld, ANDES and most of the industry.

3.1 What they are

GENCLS — the *classical* model. Two states, δ and ω , with a constant voltage $E \angle \delta$ behind a fixed transient reactance $X'd$. No field winding, no dampers, so an AVR or PSS cannot be attached to it in any meaningful way. Used for machines far from the disturbance, and for system equivalents.

GENROU — the *round-rotor* model, the workhorse for thermal units. Six states: δ , ω , $E'q$, $E'd$, $\psi''d$, $\psi''q$. One field winding and one damper on the d-axis; two windings on the q-axis. Quadratic saturation on the air-gap flux. Its sibling **GENSAL** is the salient-pole version (no q-axis transient winding, d-axis saturation only); **GENTPF/GENTPJ** are newer variants with better saturation, GENTPJ's depending on stator current too.

3.2 How they differ from the model you have

There are four differences, and only two of them are real.

(a) Parameter form — the big one. Your model is written in *circuit* parameters: actual winding inductances and resistances, L_{ad} , L_{aq} , L_{fd} , R_{fd} , L_{1d} , R_{1d} , L_{1q} , R_{1q} , L_{2q} , R_{2q} , with the currents recovered by inverting a 3×3 inductance matrix per axis:

```
invd = Matrix([[-Lad-Ll, Lad, Lad], [-Lad, L1d+Lad, Lad], [-Lad, Lad, Lfd+Lad]]).inv()
```

GENROU is written in *operational* parameters — X_d , X_q , $X'd$, $X'q$, $X''d$, $X''q$, X_l , $T'do$, $T'qo$, $T''do$, $T''qo$ — which is the form manufacturers publish and every data file contains. The two are related by a standard but assumption-laden conversion. **This is the practical difference: your model cannot ingest a PSS/E .dyr file without that conversion.**

(b) Stator transients. Your full model keeps $d\psi_d/dt$ and $d\psi_q/dt$, so it represents the 50/60 Hz stator transient and the DC offset — it is an EMT model. GENROU never has them; it is a phasor/RMS model by construction. **Your order6 level already removes them**, which is the counterpart.

(c) The speed-voltage term. Your stator equations carry $\omega_r \cdot \psi$ with the actual rotor speed. GENROU (and most RMS tools) sets $\omega \approx 1$ there. Yours is the more accurate choice; the difference is second-order in speed deviation and only visible in large excursions.

(d) Saturation. GENROU has it, you do not. This is a genuine gap, and it matters at full load and during faults, where the effective X_d drops noticeably.

(e) Subtransient saliency. Classic PSS/E GENROU enforces $X''d = X''q$ (a modelling convenience, not physics). Your $L_{1q}/L_{2q}/L_{aq}$ are free, so you do not have that restriction. ANDES parameterises x_{d2} and x_{q2} separately too.

3.3 The mapping

PSS/E	G2ELin level	Honest verdict
GENCLS	order2 — swing, field flux frozen	Equivalent. Constant flux behind $X'd$ is exactly the classical model, and freezing rather than residualising is the correct choice.
GENROU	order6 — stator flux algebraic, all four rotor windings dynamic	Structurally the same model. Same winding count, same neglect of stator transients. Differs by: parameter form, no saturation, ω_r kept in the stator equations, $X''d \neq X''q$ allowed.

PSS/E	G2ELin level	Honest verdict
GENSAL	order5 (drop damper_2q)	Close. GENSAL's defining assumption is $X'q = Xq$.
—	full (19 states)	No PSS/E equivalent. Stator transients put this outside what any phasor tool represents; the comparison is with an EMT tool (PSCAD, EMTP, Simscape).

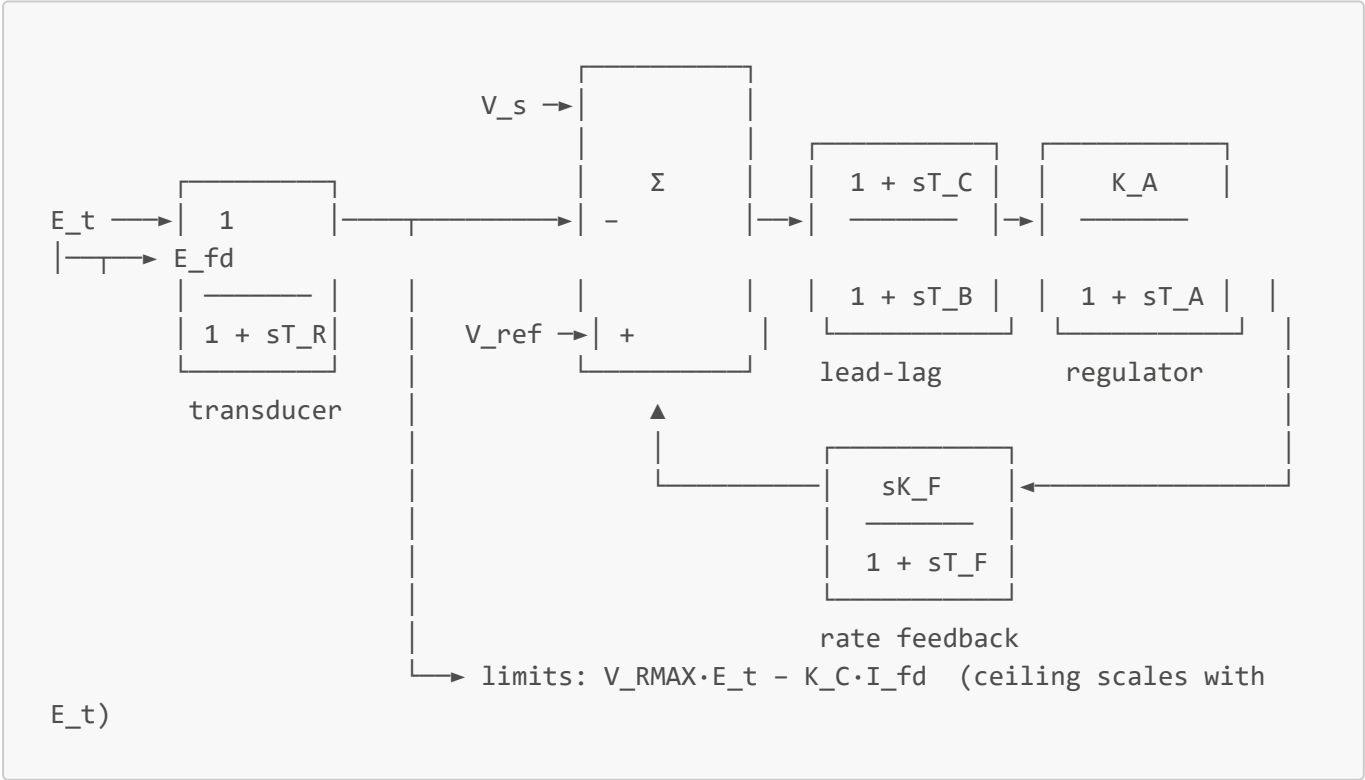
So the short answer: **you have already implemented GENROU, in a different parameter set, without saturation, and with an EMT mode on top that PSS/E does not have.** The two things worth adding for real interoperability are the operational-parameter conversion and saturation — both separate from the controller work in this document.

4. The proposed models

Notation: E_t terminal voltage, V_{ref} reference, V_s stabiliser output, E_{fd} field voltage, ω rotor speed pu, P_m mechanical power. Limits are shown in the diagrams and listed per model; §5.1 covers how they should be treated.

4.1 EXST1 / ST1A — static (thyristor) exciter · 4 states

This is the model in your figure. Potential-source, controlled-rectifier: the exciter *is* a thyristor bridge fed from the generator terminals, so there is no exciter time constant at all — the field voltage follows the regulator directly, and the ceiling is proportional to E_t .



$$\begin{aligned} dv_1/dt &= (E_t - v_1) / T_R \\ u &= V_{ref} - v_1 - v_F + V_s \\ &\quad V_{IMIN}..V_{IMAX} \\ dv_2/dt &= (u - v_2) / T_B \end{aligned}$$

(limited to

```

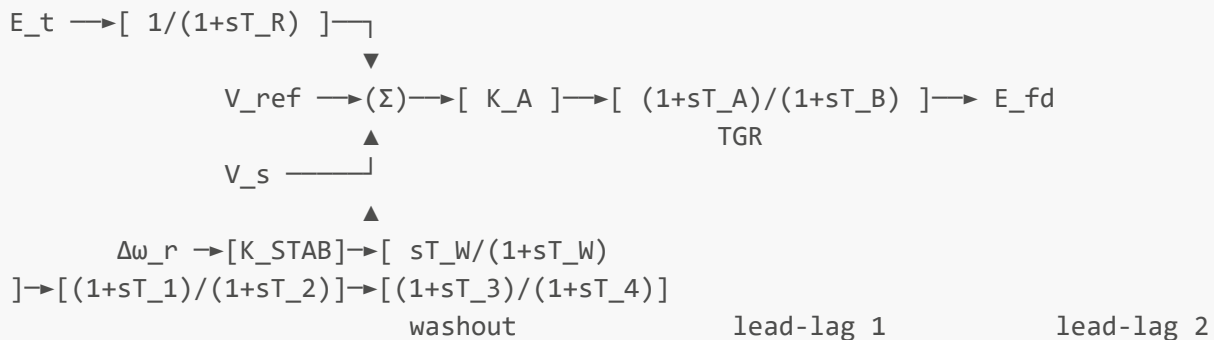
y      = (T_C/T_B)·u + (1 - T_C/T_B)·v_2      lead-lag output
dv_R/dt = (K_A·y - v_R) / T_A
dv_F/dt = (K_F·(dv_R/dt) - v_F) / T_F
E_fd    = v_R                                limited to V_RMIN..V_RMAX (×E_t,
-K_C·I_fd)

```

Parameters: T_R , V_{IMAX} , V_{IMIN} , T_C , T_B , K_A , T_A , V_{RMAX} , V_{RMIN} , K_C , K_F , T_F .

Your Kundur figure as a special case

The figure is EXST1 with the regulator lag removed and no rate feedback:



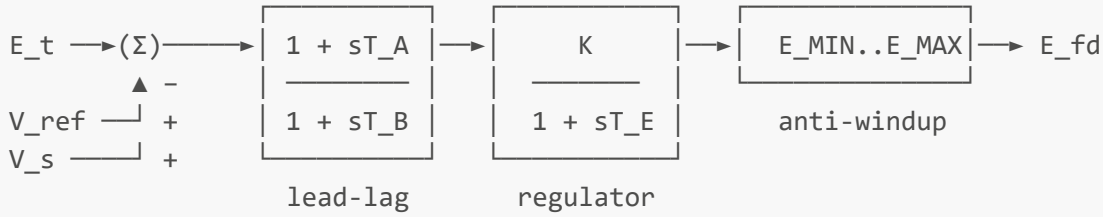
Two traps in this figure, both about names.

1. T_A and T_B mean different things than in IEEE ST1A. Kundur's TGR block is $(1+sT_A)/(1+sT_B)$; IEEE's lead-lag is $(1+sT_C)/(1+sT_B)$ and IEEE's T_A is the *regulator lag*, which Kundur's figure does not have. Mapping: $T_C := T_A^{Kundur}$, $T_B := T_B^{Kundur}$, $T_A^{IEEE} = 0$, $K_F = 0$. If the parameter names are taken straight off the figure into an EXST1 implementation, the lead time constant lands in the regulator lag and the model is silently wrong. **The catalogue should store IEEE names and offer the Kundur figure as a named preset that fills them.**

2. The summing-junction signs as drawn are positive feedback. The screenshot marks E_t with + and V_{ref} with -, giving $E_{fd} = K_A \cdot (E_t - V_{ref} + V_s) \cdot TGR$. With $K_A > 0$ a rise in terminal voltage would *raise* field voltage. A regulator must be negative feedback, so the implementable form is $E_{fd} = K_A \cdot (V_{ref} - E_t + V_s) \cdot TGR$ — which is what every other Kundur figure and IEEE 421.5 use. **Worth confirming against the book before transcribing**, but do not implement the signs as drawn.

TGR (transient gain reduction) is the point of the T_A/T_B block: with $T_B \gg T_A$ the steady-state gain stays at K_A while the gain above $1/T_B$ falls to $K_A \cdot T_A/T_B$. High DC gain gives tight voltage regulation; low transient gain stops the AVR from destabilising the electromechanical mode. A PSS is the alternative way to buy the same stability, which is why the two appear together in this figure — and why this single figure is the right first model to implement: **it is the textbook demonstration of the AVR-destabilises / PSS-restores interaction your eigenvalue map exists to show.**

4.2 SEXS — simplified excitation system · 2 states



```

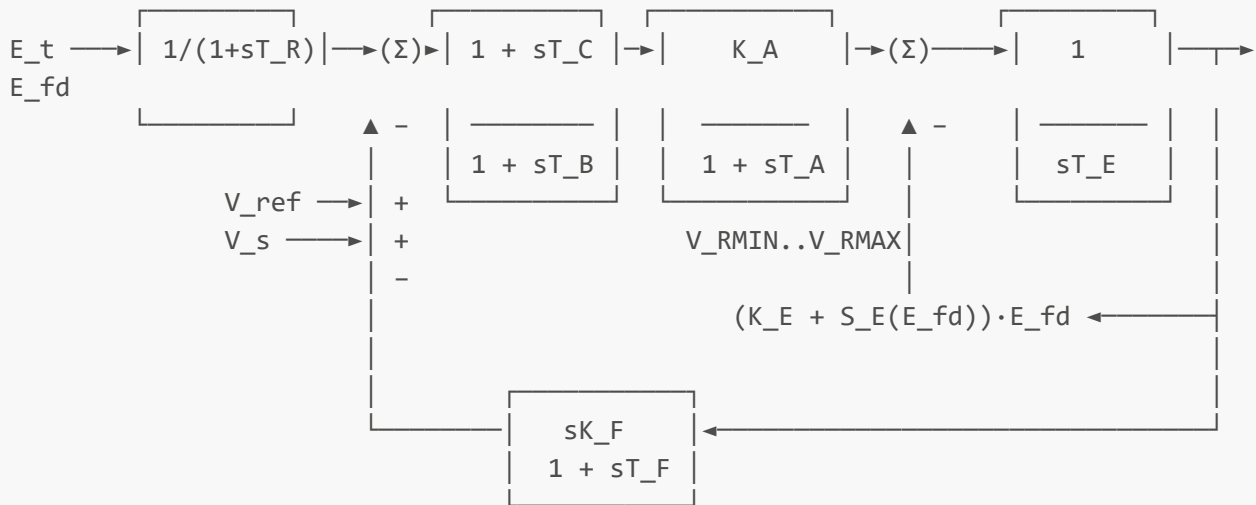
u      = V_ref - E_t + V_s
dx_1/dt = (u - x_1) / T_B
y      = (T_A/T_B)·u + (1 - T_A/T_B)·x_1
dE_fd/dt = (K·y - E_fd) / T_E      anti-windup at E_MIN..E_MAX

```

Parameters: T_A/T_B (ANDES takes the *ratio* $TATB$ and TB), K , T_E , E_{MIN} , E_{MAX} . No terminal-voltage transducer lag — the measurement is assumed instantaneous.

Two states, no saturation, no feedback loop. This is what to attach to the forty machines in a 118-bus case that you have no real data for; it is stable and well-behaved and does not invent detail.

4.3 EXDC2 / IEEE1 — DC commutator exciter · 4 states



```

dv_1/dt = (E_t - v_1) / T_R
u      = V_ref - v_1 - v_F + V_s
dx_LL/dt = (u - x_LL) / T_B ;   y = (T_C/T_B)·u + (1 - T_C/T_B)·x_LL
dv_R/dt = (K_A·y - v_R) / T_A      anti-windup V_RMIN..V_RMAX
dE_fd/dt = (v_R - (K_E + S_E(E_fd))·E_fd) / T_E
dv_F/dt = (K_F·(dE_fd/dt) - v_F) / T_F

```

with **quadratic saturation** (ANDES `ExcQuadSat`, from the two published points (E_1, S_{E1}) and (E_2, S_{E2})):

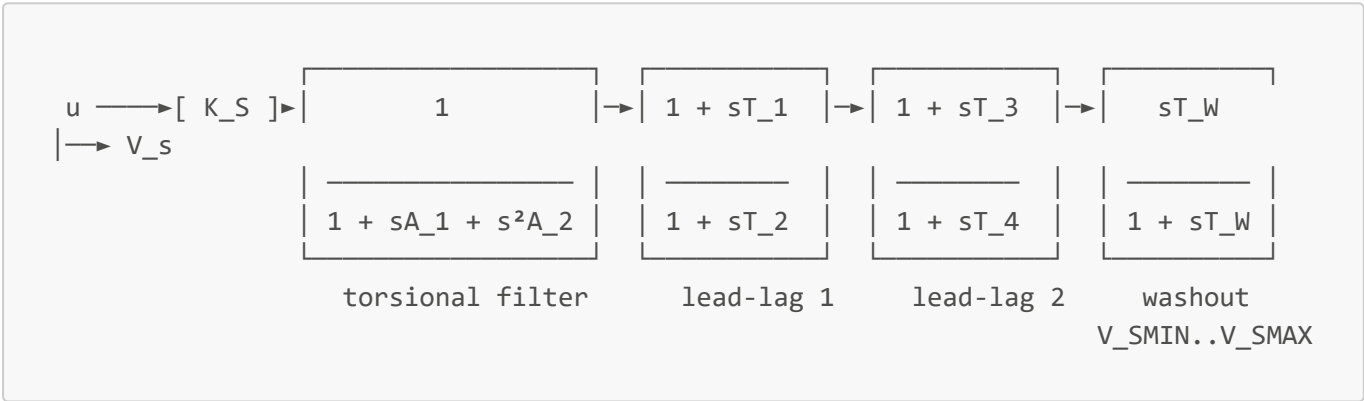
$$S_E(x) = B \cdot (x - A)^2 / x \quad \text{for } x > A, \quad 0 \text{ otherwise}$$

Parameters: T_R , K_A , T_A , T_B , T_C , V_{RMAX} , V_{RMIN} , K_E , T_E , K_F , T_F , E_1 , S_{E1} , E_2 , S_{E2} .

This is the closest standard model to what G2ELin has today. Implementing it converts the existing ad-hoc exciter into a named, documented, validatable one, and the migration is mostly a parameter remapping — plus the K_e correction in §2.3 and the saturation term, which is new.

S_E is smooth and differentiable everywhere except at $x = A$, so it can go into the symbolic model directly and be linearised; no special treatment is needed as long as the operating point is not sitting exactly on the breakpoint.

4.4 PSS1A — single-input stabiliser · 5 states



G2ELin's present PSS is this with $1/(1+sA_1+s^2A_2)$ collapsed to $1/(1+sT_{LP})$, and with the washout placed first rather than last (identical, the chain is linear). **So this is a two-state addition plus limits**, not a new model.

The second-order filter is what rejects the **torsional modes** of the turbine-generator shaft (typically 10–50 Hz). A stabiliser with enough gain to damp a 0.2–2 Hz electromechanical mode will otherwise excite them, which is a real and destructive failure mode. A single pole cannot place a complex-conjugate notch; A_1/A_2 can.

4.5 IEEEEST — generalised single-input stabiliser · 7 states

PSS1A plus a *selectable input signal* and a second-order lead-lag. Worth having because the choice of input is itself a design question your tool is well placed to answer:

MODE	Input signal
1	rotor speed deviation $\omega - 1$
2	bus frequency deviation
3	generator electrical power
4	accelerating power $P_m - P_{m0}$

MODE	Input signal
5	bus voltage magnitude
6	bus voltage derivative

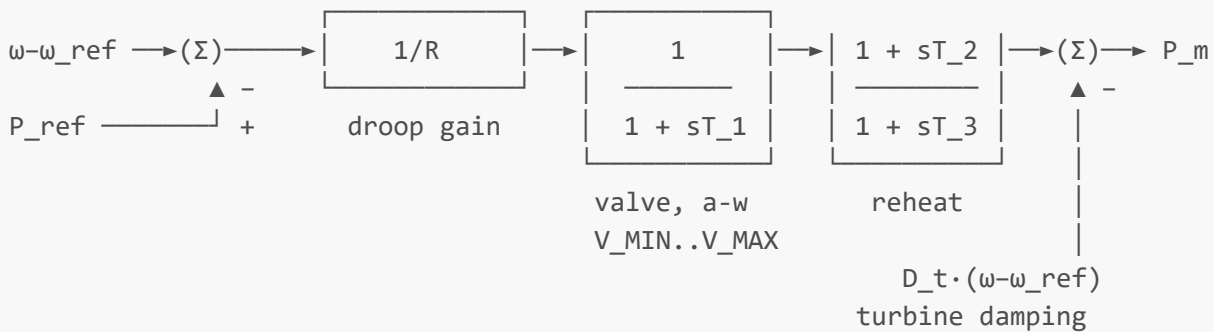
```

u → [ 1/(1+sA_1+s^2A_2) ] → [ (1+sA_3+s^2A_4)/(1+sA_5+s^2A_6) ]
    → [ (1+sT_1)/(1+sT_2) ] → [ (1+sT_3)/(1+sT_4) ] → [ K_S ]
    → [ sT_5/(1+sT_6) ] → [ L_SMIN..L_SMAX ] → V_s      (gated off outside
V_CL..V_CU)

```

If you implement only one stabiliser, implement this one — PSS1A is the special case with $A_3..A_6 = 0$.

4.6 TGOV1 — steam governor + turbine · 2 states



```

w_d      = w - w_ref
p_d      = (P_ref - w_d) / R
dx_1/dt  = (p_d - x_1) / T_1
dx_2/dt  = (x_1 - x_2) / T_3
y        = (T_2/T_3) * (x_1 - x_2) + x_2
P_m      = y - D_t * w_d

```

anti-windup V_MIN..V_MAX

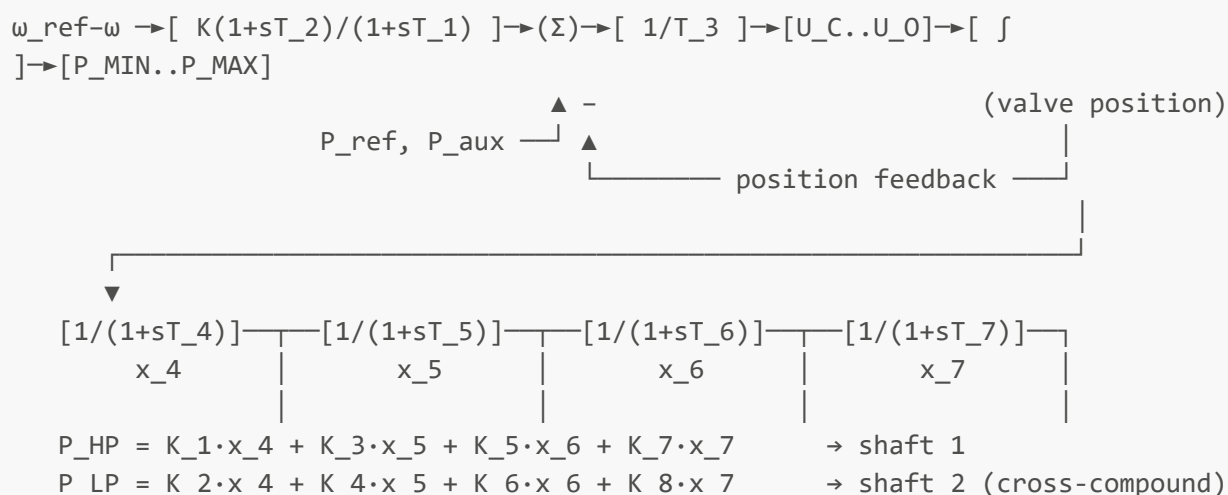
Parameters: R , T_1 , T_2 , T_3 , D_t , V_{MAX} , V_{MIN} .

G2ELin's current governor is exactly the x_1 stage, with $R = m_p$, $T_1 = T_G$, $T_2 = T_3$ (lead-lag transparent) and $D_t = 0$. So TGOV1 is one extra state, one extra parameter and a limit — the cheapest real model on this list and the one I would do first.

T_2/T_3 represents the **reheater**: the fraction of power in the HP stage appears fast, the rest arrives with the reheat time constant T_3 (5–10 s). It is why steam units cannot arrest a frequency excursion as fast as their valve moves, and it is invisible in the current model.

4.7 IEEEG1 — detailed steam governor · 6 states

Tandem-compound turbine with four steam volumes and up to eight power take-off fractions, optionally driving *two* generators (cross-compound).

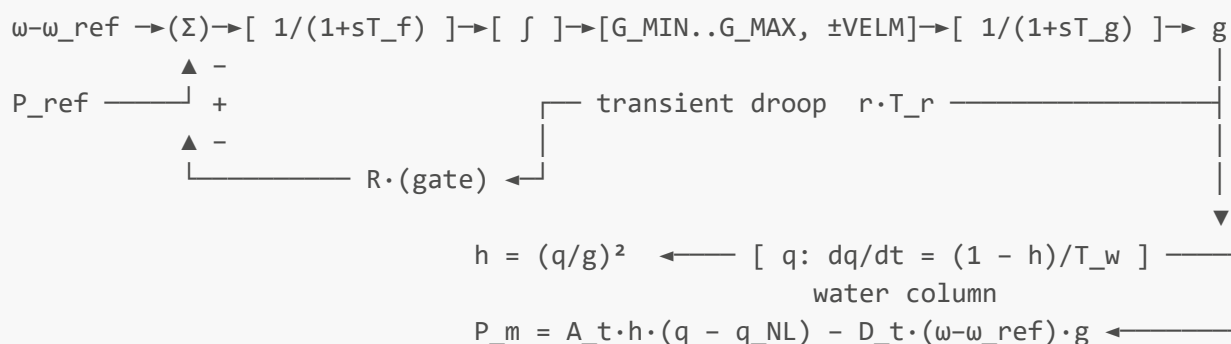


$w_d = \omega_{ref} - \omega$
 $dx_{LL}/dt = (w_d - x_{LL})/T_1 ; y_{LL} = K \cdot [(T_2/T_1)(w_d - x_{LL}) + x_{LL}]$
 $v_s = (y_{LL} + P_{ref} + P_{aux} - x_I)/T_3$ valve rate, limited $U_C..U_0$
 $dx_I/dt = v_s$ anti-windup $P_{MIN}..P_{MAX}$
 $dx_4/dt = (x_I - x_4)/T_4$ steam chest
 $dx_5/dt = (x_4 - x_5)/T_5$ reheater
 $dx_6/dt = (x_5 - x_6)/T_6$ crossover
 $dx_7/dt = (x_6 - x_7)/T_7$ second reheat
 $P_m = K_1 \cdot x_4 + K_3 \cdot x_5 + K_5 \cdot x_6 + K_7 \cdot x_7$

The $K_1..K_8$ must sum to 1 (ANDES normalises them if they do not). U_C and U_0 are **rate** limits on the valve — closing and opening velocity — which is a different kind of limit from all the others here and the one most likely to bind during a real frequency event.

4.8 HYGOV — hydro governor with penstock · 4 states

The only genuinely nonlinear model on the list, and the reason it belongs here: a hydro unit that is asked for more power first delivers **less**, because opening the gate drops the head before the flow builds. A linear governor cannot produce that, and it changes the sign of the early frequency response.



```

p_d      = P_ref - (w - w_ref) - R.g_desired
dx_f/dt  = (p_d - x_f)/T_f
dc/dt    = x_f                                     gate position integrator,
                                                    G_MIN..G_MAX, rate ±VELM

g_desired= c + x_f/r
dg/dt    = (g_desired - g)/T_g                     gate servo
dq/dt    = (1 - (q/g)2)/T_w                         water column ← the nonlinearity
P_m      = A_t·(q/g)2·(q - q_NL) - D_t·(w-w_ref)·g

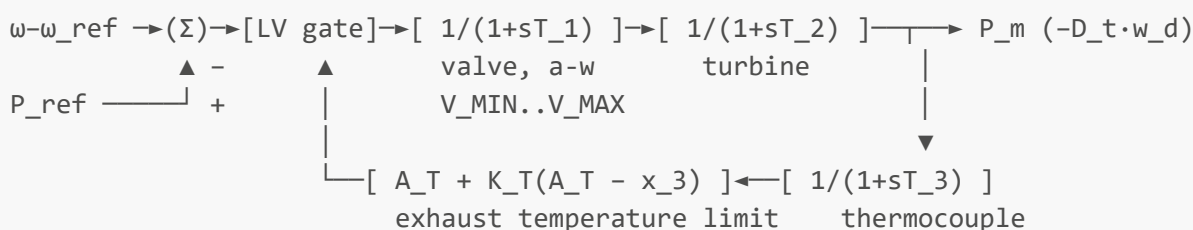
```

Parameters: R , r , T_r , T_f , T_g , V_{ELM} , G_{MIN} , G_{MAX} , T_w , A_t , D_t , q_{NL} .

T_w (water starting time, 0.5–4 s) sets the right-half-plane zero. r/T_r is the **transient droop** — a temporary droop much larger than the permanent one, which is the classical fix for exactly this non-minimum-phase behaviour.

$(q/g)^2$ is differentiable for $g > 0$, so this linearises fine; only the gate limits need the §5.1 treatment.

4.9 GAST — gas turbine · 3 states



```

w_d      = w - w_ref
p_d      = (P_ref - w_d)/R
v_9      = A_T + K_T·(A_T - x_3)                   temperature-limit demand
dx_1/dt  = (min(p_d, v_9) - x_1)/T_1               anti-windup V_MIN..V_MAX
dx_2/dt  = (x_1 - x_2)/T_2
dx_3/dt  = (x_2 - x_3)/T_3
P_m      = x_2 - D_t·w_d

```

The **low-value gate** $\min(p_d, v_9)$ is the exhaust-temperature limiter, and it is what makes a gas turbine's frequency response asymmetric: it can shed load freely but cannot pick it up past the temperature ceiling. It is also a $\min()$, i.e. non-differentiable — see §5.1.

5. Decisions to make before implementing

5.1 Limits, gates and dead-bands — the central question

Every model above has them; G2ELin has none. They are not differentiable, and G2ELin's whole pipeline is a symbolic Jacobian.

The standard resolution, which I recommend:

- **For linearisation: ignore them, but check them.** Small-signal analysis is about behaviour *around* an operating point, and at a valid operating point no limit should be binding. The right move is not to silently drop them but to evaluate every limit at the operating point and **emit a warning when one is active** — a machine sitting on its `V_RMAX` has no AVR loop at all, and its eigenvalues are meaningless if the limit is ignored. This fits the existing `notes` mechanism (`modal/adequacy.py` already does this kind of reporting).
- **For time-domain: implement them properly**, with anti-windup on every integrator behind a limit. The nonlinear path (`nonlinear_funcs`) does not need differentiability, and `timedomain/emt.py` already uses a Newton solve that can carry a saturated branch. Note that hard limits will hurt the variable-step solvers (Radau et al.) unless they are smoothed or handled as events — this needs a decision of its own.
- **Dead-bands and low-value gates** (`GAST`, `IEEEG1`, `HYGOV`) are the same problem in a sharper form. `min/max` can be linearised by picking the active branch at the operating point, which is exactly the "ignore but check" rule above.

This decision should be made before any model is written, because it determines whether a limit is a parameter carried through the symbolic model or a wrapper around it.

5.2 The reduction catalogue becomes model-dependent

`reduction.py` currently hard-codes the state groups:

```
StateGroup("pss", "Power system stabiliser", ("Dw1", "v1", "v2", "vpss"), ...)
StateGroup("avr", "AVR / exciter", ("e1", "e2", "efd", "e3"), ...)
StateGroup("governor", "Governor", ("Pm",), ("P_m",), ...)
```

With selectable controllers, the symbols in these groups depend on which model the unit has. `ElementModel` needs to become a function of the controller choice, or the groups need to be contributed by the controller block itself.

This is the identical problem to the deferred GFM controller work (dVOC, VSM and Matching add and remove states relative to Droop, and VSM/Matching drop `pm/qm` entirely). The two should be designed together and the refactor done once. I would treat that as a prerequisite rather than a parallel task.

5.3 A block architecture

`sm_dae()` currently builds all nineteen equations inline. I propose factoring each controller into a function returning a small bundle:

```
@dataclass(frozen=True)
class ControlBlock:
    states: list[sp.Symbol]
    diffeqs: list[sp.Expr]
    state_names: list[str]
    groups: tuple[StateGroup, ...] # its own contribution to the catalogue
    output: sp.Expr # V_s, or E_fd, or P_m
    params: dict[str, float] # its defaults
```

```
def exciter_block(kind: ExciterModel, *, e_t, v_ref, v_s) -> ControlBlock: ...
def pss_block(kind: PssModel, *, dw_r, p_e, v_t) -> ControlBlock: ...
def governor_block(kind: GovernorModel, *, omega, w_ref, p_ref) -> ControlBlock:
...

```

`sm_dae(is_slack, modes, exciter, pss, governor)` then concatenates them. The `lru_cache` key extends naturally. The existing lead-lag idiom (§2.2) gives a shared helper:

```
def lead_lag(u, du, T_n, T_d, state): # returns dstate/dt
    return (T_n * du + u - state) / T_d

```

Parameter naming needs a decision too: `sm_params()` is currently one flat dict, and `EXST1.T_A` would collide with nothing today but `TGOV1.T_1` and `IEEEG1.T_1` collide with each other. Either prefix by block (`exc_TA`, `gov_T1`) or nest the dict. **Prefixing is less disruptive** — `DerUnit.params` is flat, `overridable_param_keys()` assumes flat, and the parameter-sensitivity scan (`modal/parameters.py`) reports flat names that users read.

5.4 Schema, API and UI

`DerUnit.controller: GfmController | None` is the precedent. Add:

```
exciter: ExciterModel = ExciterModel.G2ELIN_LEGACY
pss: PssModel = PssModel.G2ELIN_LEGACY
governor: GovernorModel = GovernorModel.G2ELIN_LEGACY

```

Keeping the present models as explicitly-named `legacy` members (rather than silently remapping them onto `EXDC2/PSS1A/TGOV1`) is what preserves every saved network and every golden test — including the MATLAB parity suite, which must keep reproducing `symSG.m` exactly. New networks can default to the standard models; old ones keep what they had.

The UI work is the model-order panel (`web/js/model_order.js`) gaining three selectors per unit, and the parameter editor becoming model-dependent.

5.5 Validation

Each model gets a test against ANDES's implementation of the same model, in the shape `tools/compare_andes.py` already establishes: build the same single-machine system in both, compare eigenvalues and a step response. This is the strongest argument for the shortlist in §1 — nine of the twelve models are checkable this way, on a package already in the environment.

Where ANDES has no counterpart (PSS2B), the alternative is a transfer-function check: excite the block alone and compare its frequency response against the analytic $H(s)$ from the standard. Weaker, but not nothing.

6. Suggested phasing

Phase 0 — prerequisites. Decide §5.1 (limits). Do the §5.2 refactor, together with the deferred GFM controller types. Add the `ControlBlock` plumbing of §5.3 and the schema fields of §5.4, with the present models wrapped unchanged as `legacy`. *No new physics — the whole test suite must still pass unchanged.*

Phase 1 — the three that are supersets. `TGOV1`, `PSS1A`, `EXDC2`. Each is a small extension of something that already works, so they exercise the new architecture without much new risk. `EXDC2` carries the `Ke` correction from §2.3 and the first saturation function.

Phase 2 — the ones that add capability. `EXST1` (with the Kundur figure as a named preset, and its parameter-name trap documented), `SEXS`, `IEEEST`, `HYGOV`, `GAST`, `IEEEG1`. `HYGOV` is the one that will exercise the nonlinear path hardest.

Phase 3 — later, on request. `PSS2B` (no ANDES partner; the integral-of-accelerating-power construction and its ramp-tracking filter $(1+sT_8)^N/(1+sT_9)^M$ need transcribing from IEEE 421.5 Fig. 8 directly, and the exact block interconnection is easy to get subtly wrong — it should be done against a paper copy of the standard, not from memory). `GGOV1`, `ESST4B`, `AC8B`, `PSS4B`.

Separately, and not in this document's scope: machine saturation and the operational-parameter (`Xd`, `X'd`, `T'do`, ...) ingestion path from §3.2. Those are what would let G2ELin read a PSS/E `.dyr` file, which is a bigger prize than any single controller model and should be planned on its own.

7. Open questions for you

1. **Limits in the linear model** — is "ignore but warn when binding" the behaviour you want, or would you rather refuse to linearise a machine sitting on a limit?
2. **Parameter namespacing** — flat with block prefixes (`exc_TA`, `gov_T1`), or nested per block? The flat choice keeps `modal/parameters.py` and the override mechanism untouched.
3. **Defaults for new networks** — keep `legacy` everywhere so nothing moves, or default new units to `EXST1/PSS1A/TGOV1`?
4. **Is the shortlist right for your users?** It is biased toward thermal and hydro plant, which is what a transmission planner wants. Given G2ELin's focus on converter-dominated systems, an argument could be made for spending phase 2 on the deferred GFM controllers instead, and treating synchronous-machine controls as a phase-1-only concern.
5. **The figure's summing-junction signs** (§4.1) — can you confirm against the book?